Mini-OpenClaw

Local-first AI agents you can actually trust.

GROUP 09 Applied Generative AI — course project

Held Johannes
11705340

Ibrahim Ahmad
11826186

Ibrar Muhammad
12350094

Bourbigua Manal
12347522

Abstract

Mini-OpenClaw is a local-first AI agent that turns plain-language requests into safe, auditable actions on your own machine. Ask it to **organise and search files**, **read and edit documents**, **run shell commands**, **fetch live web data**, **remember facts across sessions**, **schedule recurring jobs**, or **delegate multi-step tasks** to sub-agents — each behind a real-time approval gate and a full audit trail.

Under the hood a hybrid **Plan → ReAct → Replan** loop reasons step by step and revises its plan when an action fails, while a policy engine classifies every call as *safe*, *approval-required*, or *forbidden* before it runs. The result is a small but complete agent — autonomous enough to be useful, constrained enough to trust — running locally through swappable Anthropic, Gemini, or Ollama providers.

The problem / initial situation

General-purpose LLM agents are powerful but hard to trust: they take consequential actions — writing files, running shell commands, fetching the web — with little visibility into what was decided or why, and most depend on remote APIs that cost money and leak data. We wanted an agent we could read end to end, run offline at zero cost, and inspect after every run. The challenge was to stay autonomous enough to finish multi-step tasks while making each tool call explicit, permission-checked, and reversible.

What it delivers

Intent → tool routing

A planner turns plain language into a validated JSON plan. The LLM proposes; code decides.

Auditable memory

Five memory layers (vector + keyword) and an append-only audit log — every decision inspectable.

Extensible skills

Manifest-driven registry: add a tool by adding a module — no core-loop changes.

Safe execution

A policy engine gates every action as safe, approval-required, or forbidden before it runs.

13 + MCP
tools

3
LLM providers tested

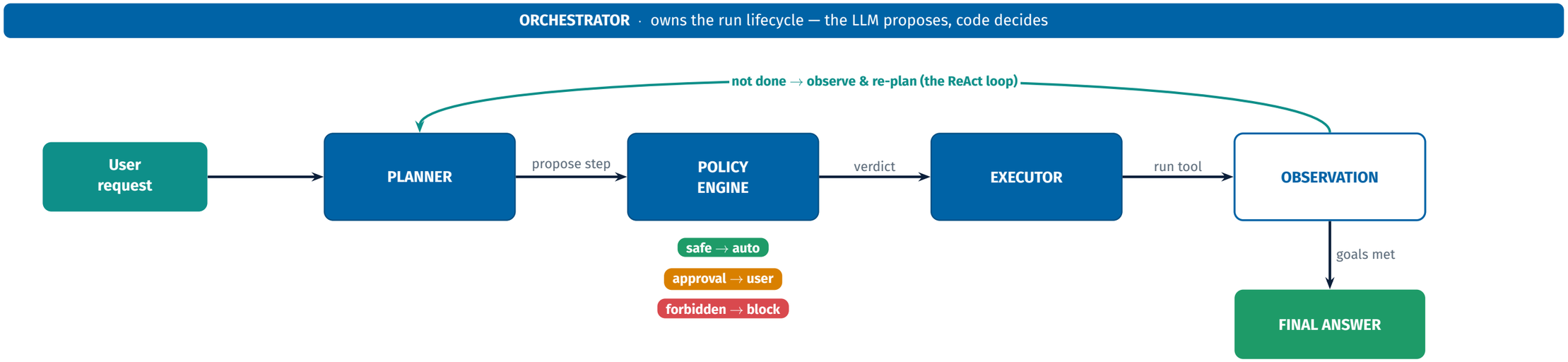
5
memory layers

TECH STACK
Backend: FastAPI · SQLite
Frontend: React + TypeScript (Vite) · SSE

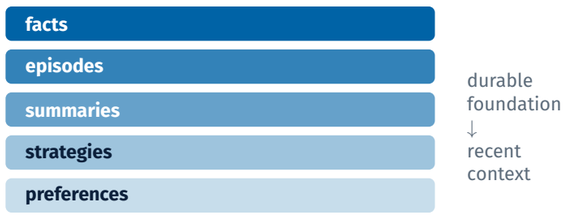
HOW IT WORKS

Approach

Every request runs as one auditable loop. The **planner** proposes a single next step; the **policy engine** classifies it as safe, approval-required, or forbidden; the **executor** runs it; and the **observation** feeds back so the planner can continue, adapt, or replan. When the goal is met the loop returns a **final answer**. Around it, **memory** supplies context, the **audit log** records every decision, and **SSE** streams progress to the UI.



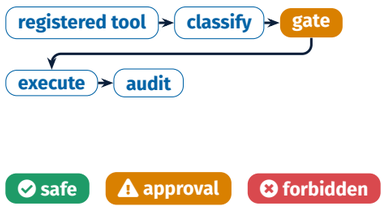
Five-layer memory



Hybrid retrieval blends 70% vector (local `all-MiniLM-L6-v2`, $0) + 30% keyword, injected into the planner each call.

Agent Dreams consolidate runs into proposed strategies under *propose → review → approve* — never acting on inferred knowledge without consent.

Safe by design



Only registered tools with valid JSON schemas run. The policy engine enforces workspace path limits, shell allow-lists, and prompt-injection checks.

Risky writes, shell, web-fetch, and delegation need explicit approval — and approval is invalidated if the payload changes.

Providers & tools

Anthropic Claude · Gemini · Ollama — local, $0

13 TOOLS (∗ = approval-gated)

`list_files` · `read_file` · `write_file∗` · `search_in_files` · `run_shell_safe∗` · `remember_fact` · `search_memory` · `delegate_task∗` · `fetch_url∗` · `schedule_task` · `get_datetime` · `calculator` · `system_info`

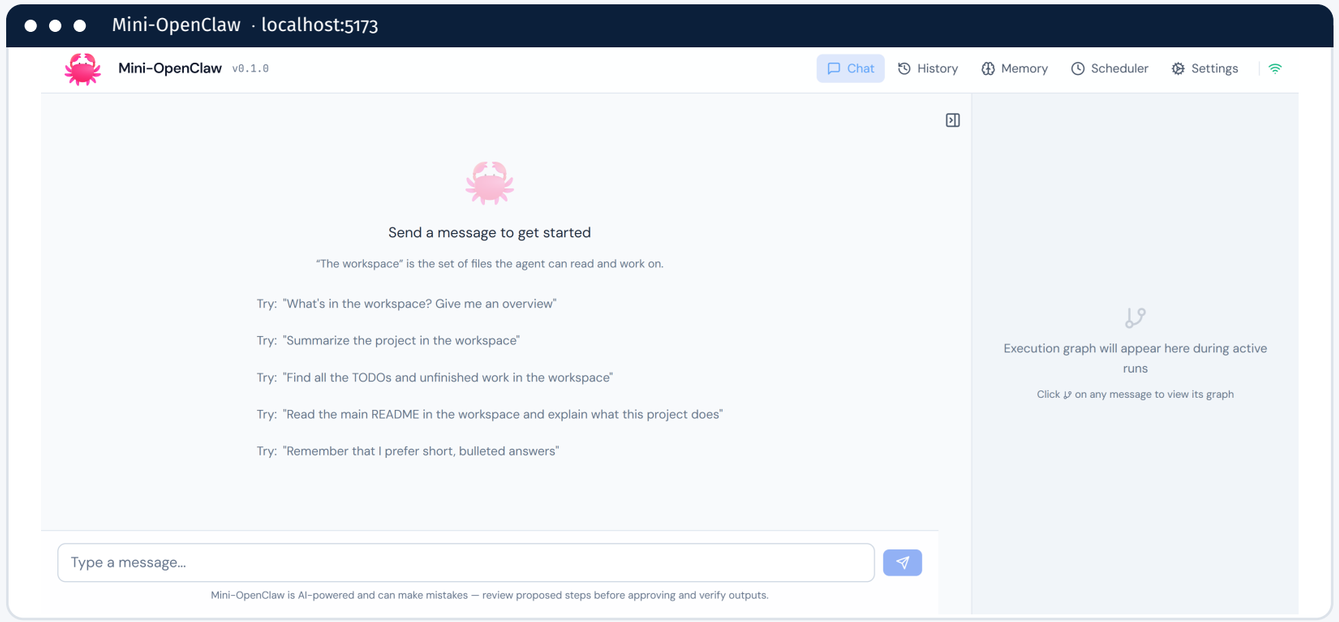
+ **(explain_run · mcp_tool∗)**

A manifest declares each tool's schema, risk level, and approval flag. The registry discovers tools at startup — the planner only sees registered tools, and adding one needs no core changes.

MCP interop: act as *client* (consume external tools) or *server* (expose tools to Claude Desktop and others). Default off; proxy tools gated at HIGH risk.

WHAT IT DOES TODAY

Results & Reflection

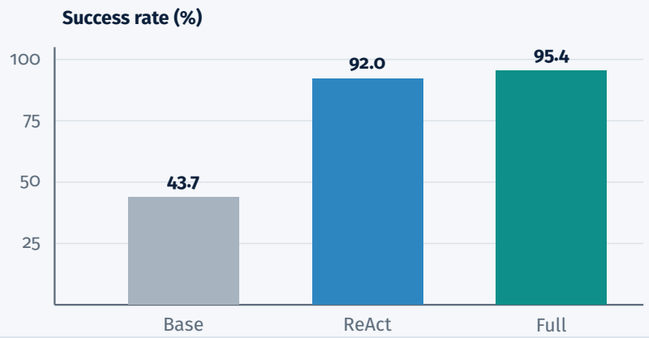


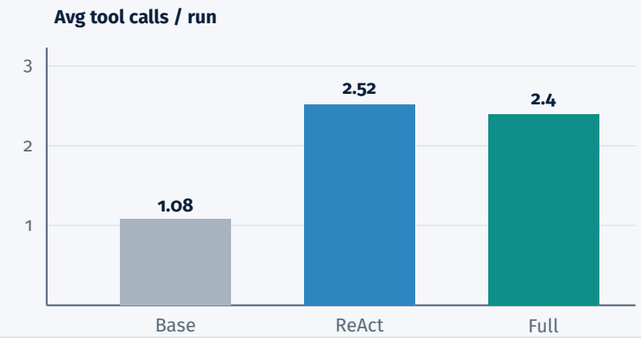
Capability success (%)

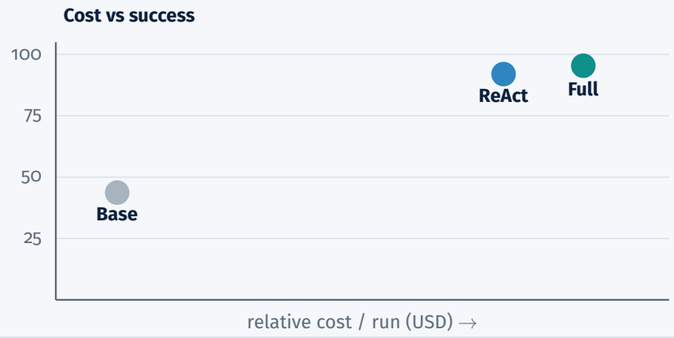
	Baseline	Plan+ReAct	Full
recall	54.2	95.8	100
search	41.7	75	75
completeness	100	100	100
multi-step	27.8	94.4	100
recovery	0	80	100
cross-file	100	100	100
memory	100	100	100
delegation	50	100	100

Where the pipeline earns its cost

Method: 14 deterministic tasks across 8 capability categories, run in a sandboxed fixture workspace and scored by deterministic verifiers (file contents, keyword presence, numeric answers) — no LLM-as-judge. Across the suite (Baseline plan-and-execute vs. Plan+ReAct vs. the Full hybrid), the full loop spends more tool calls and money per run but wins clearly on **completeness**, **recovery**, and **multi-step** tasks.







What worked well

- Separating planning / policy / execution / memory / logging made the agent debuggable.
- Policy engine + approval gates gave safety without crippling capability.
- The hybrid loop improved completeness and recovery over a plain plan-and-execute baseline.
- Local embeddings give useful semantic memory at no extra cost ($0).

Challenges

- Naive ReAct can stall or duplicate calls — capped with duplicate detection + budgets.
- Real approval-friction vs. autonomy trade-off — too many gates slow the user.
- LLM variance makes evaluation non deterministic — fails in one run can vanish in the next.
- The evaluation suite is small and illustrative, not a rigorous benchmark.

Further work

- Replace binary approval with time-bounded, extendable per-path / per-tool permissions.
- Move beyond single-user local use (SQLite, no auth) to multi-user with auth., per-user workspaces, and isolated memory.
- Add memory eviction / decay so the three-layer store stays relevant and bounded over long-lived use.
- Broaden tool coverage with new capabilities (structured data, database, third-party APIs) each gated by the policy engine.